

Project Report Format

Script-Controlled ACL – Restrict Record Access Based on Field Value

1. INTRODUCTION

1.1 Project Overview: A ServiceNow security project that restricts access to records in the custom u_institution_details table using a script-controlled Read ACL, a Branch = EEE condition, and role-specific Create, Write, and Delete ACLs.

1.2 Purpose: Protect record-level data, allow EEE users to see only EEE records, retain full administrator access, and control record lifecycle actions using bb1, bb2, bb3, and bb4.

2. IDEATION PHASE

2.1 Problem Statement: Users with broad table access may see or modify records outside their branch. The project addresses this by evaluating identity, role, operation, and Branch value together.

2.2 Empathy Map Canvas: Administrators need complete oversight; EEE users need safe access to EEE records; data owners need controlled creation and modification; testers need clear allow/deny evidence. Concerns include data leakage, accidental edits, and unauthorized deletion.

2.3 Brainstorming: Candidate approaches included role-only ACLs, condition-only filtering, scripted ACLs, and separate operation ACLs. The selected solution combines a scripted Read ACL with a Branch condition and separate role-gated Create, Write, and Delete controls.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map: Admin creates users, roles, table, fields, and records; EEE User opens the table and sees EEE records; users with bb2/bb3/bb4 perform permitted lifecycle operations; unauthorized attempts are denied; tester records evidence.

3.2 Solution Requirement: Create EEE User; roles bb1–bb4; table u_institution_details; ECE/EEE/CSE Branch choices; Read, Create, Write, and Delete record ACLs; admin override; verification in list, form, and supported Service Portal views.

3.3 Data Flow Diagram: User login → ServiceNow identity and roles → requested table operation → ACL evaluation → role/script/data-condition decision → allow authorized records/action or deny request. Test data flows through ECE, EEE, and CSE records.

3.4 Technology Stack: ServiceNow platform; System Definition custom table; User Administration and Roles; System Security record ACLs; GlideSystem role check gs.hasRole; list, form, and optional Service Portal interfaces.

4. PROJECT DESIGN

4.1 Problem Solution Fit: The design fits record-level branch security because it evaluates the record's Branch value for reads and applies operation-specific roles for lifecycle actions.

4.2 Proposed Solution: Use an advanced Read ACL on u_institution_details: allow admin; allow bb1 for the EEE data condition; deny others. Add Create/bb2, Write/bb3, and Delete/bb4 ACLs. Validate with representative ECE, EEE, and CSE records.

4.3 Solution Architecture: ServiceNow user and role layer → ACL policy layer → u_institution_details data layer → list/form/portal presentation. Security decisions occur before protected records or operations are exposed.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning: Phase 1: create users and roles. Phase 2: create table, fields, and records. Phase 3: configure and verify Read ACL. Phase 4: configure Create, Write, and Delete ACLs. Phase 5: execute functional and performance testing. Phase 6: compile documentation and final report. The supplied lab timer is 3 hours; detailed calendar dates are to be provided.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing: Measure single-user list/form response, filtered-list behavior, controlled concurrent reads, concurrent create/update/delete stability, and sustained-session behavior. Confirm no ACL bypass, data leakage, unacceptable errors, or corrupted records. Record baseline, workload, response time, error rate, and Pass/Fail after execution; actual measurements were not supplied.

7. RESULTS

7.1 Output Screenshots: Expected evidence includes ACL configuration, roles, table fields, admin all-branch view, EEE-only view, denied access, New/Edit/Delete controls, and performance results. Screenshots and final execution results: To be added after testing.

8. ADVANTAGES & DISADVANTAGES

Advantages: clear record-level control; separate permissions by operation; administrator override; supports positive and negative testing. Disadvantages: role assignments must be

maintained; scripted ACL changes require review; performance thresholds and portal behavior require environment-specific validation.

9. CONCLUSION

The project demonstrates how ServiceNow ACLs combine roles, script logic, and record field conditions to protect sensitive records and control their lifecycle. The documentation and test templates provide the basis for formal execution and sign-off.

10. FUTURE SCOPE

Automate regression tests with ServiceNow ATF; add audit reporting; expand branch policies; introduce deployment/version control for ACLs; add monitoring for denied requests and periodic access reviews.

11. APPENDIX

Source Code (if any): The supplied project contains the Read ACL script: admin users return true; bb1 users return true; all other users return false. Create, Write, and Delete ACL configurations are role-based. Full source/export: To be attached if available.

Dataset Link: No external dataset link supplied. Test data consists of disposable u_institution_details records with ECE, EEE, and CSE Branch values.

GitHub & Project Demo Link: Not supplied. ServiceNow instance URL and demonstration link: To be added by the project team.